Deep Reinforcement Learning

Bonus Unit 3: Further Topics and Research Directions

Hugging Face Deep RL Course

Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 Introduction **1**

2 (Automatic) Curriculum Learning for RL **1**
2.1 Further reading . . . 3
2.2 Author . . . 4

3 Generalization in Reinforcement Learning **4**

4 Model Based Reinforcement Learning (MBRL) **4**
4.1 Simple definition . . . 4
4.2 Academic definition . . . 4
4.3 Further reading . . . 5
4.4 Author . . . 5

5 Offline vs. Online Reinforcement Learning **5**
5.1 Further reading . . . 6
5.2 Author . . . 6

6 Decision Transformers **6**
6.1 Learn about Decision Transformers . . . 6
6.2 Train your first Decision Transformers . . . 6
6.3 Further reading . . . 6
6.4 Author . . . 7

7 RLHF **7**
7.1 Start Learning about RLHF . . . 7
7.2 Additional readings . . . 7
7.3 Author . . . 8

8 Language models in RL **8**
8.1 LMs encode useful knowledge for agents . . . 8
8.2 LMs and RL . . . 9
8.3 Further reading . . . 10
8.4 Author . . . 10

9 An Introduction to Unreal Learning Agents **10**
9.1 Case 1: I don't know anything about Unreal Engine and Beginners in Unreal Engine . . . 11
9.2 Case 2: I'm familiar with Unreal . . . 11

10 Godot RL Agents **11**
10.1 Create a custom RL environment with Godot RL Agents . . . 12
10.2 Author . . . 18

11 Brief introduction to RL documentation **18**
11.1 Motivating Reward Reports . . . 19
11.2 Capturing temporal behavior with documentation . . . 19
11.3 Contributing . . . 19
11.4 Author . . . 19

12 Interesting Environments to try **19**
12.1 DIAMBRA Arena . . . 20
12.2 MineRL . . . 21
12.3 DonkeyCar Simulator . . . 22
12.4 Starcraft II . . . 22
12.5 Author . . . 23

13 Student Works **23**
13.1 Space Scavanger AI . . . 23
13.2 Neural Nitro . . . 24
13.3 Space War . . . 24
13.4 Decision Transformers for Trading . . . 25

1 Introduction

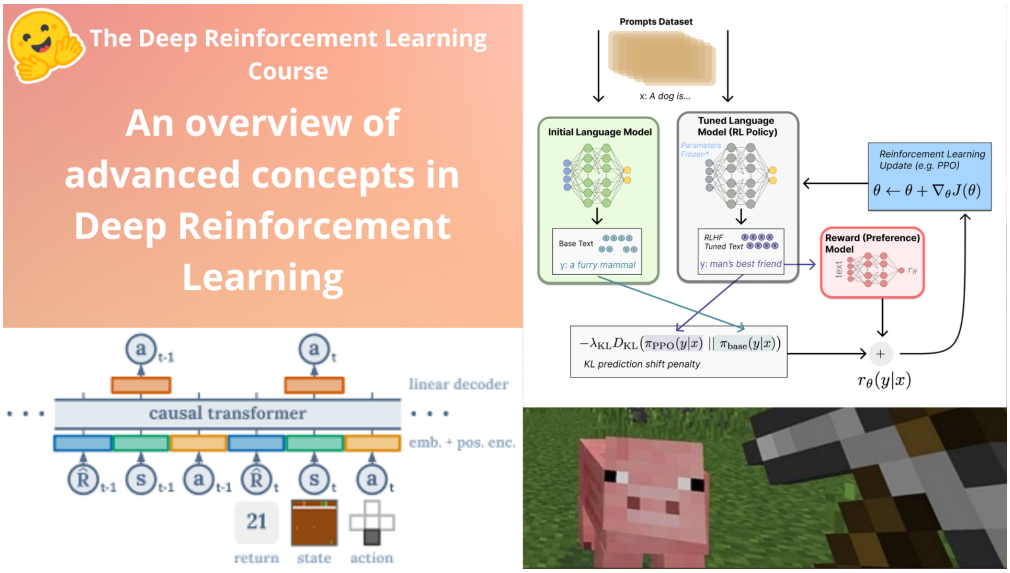


Figure 1: Hugging Face Deep RL Course: Bonus Unit 3 overview [original image]

Congratulations on finishing this course! **You now have a solid background in Deep Reinforcement Learning**. But this course was just the beginning of your Deep Reinforcement Learning journey, there are so many subsections to discover. In this optional unit, we **give you resources to explore multiple concepts and research topics in Reinforcement Learning**.

Contrary to other units, this unit is a collective work of multiple people from Hugging Face. We mention the author for each unit.

Sound fun? Let's get started ,

2 (Automatic) Curriculum Learning for RL

While most of the RL methods seen in this course work well in practice, there are some cases where using them alone fails. This can happen, for instance, when:

- the task to learn is hard and requires an **incremental acquisition of skills** (for instance when one wants to make a bipedal agent learn to go through hard obstacles, it must first learn to

stand, then walk, then maybe jump. . .)

- there are variations in the environment (that affect the difficulty) and one wants its agent to be **robust** to them

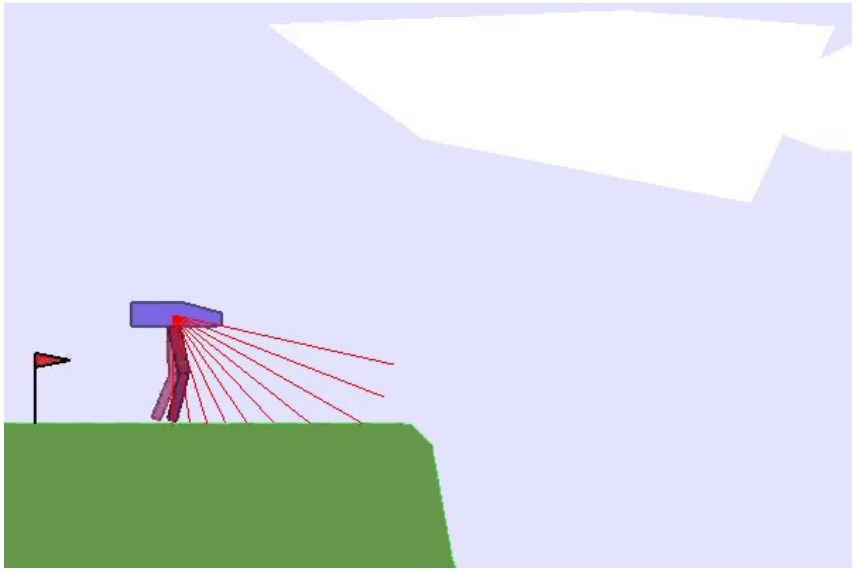


Figure 2: Bipedal locomotion in TeachMyAgent: curriculum-learning example [original image]

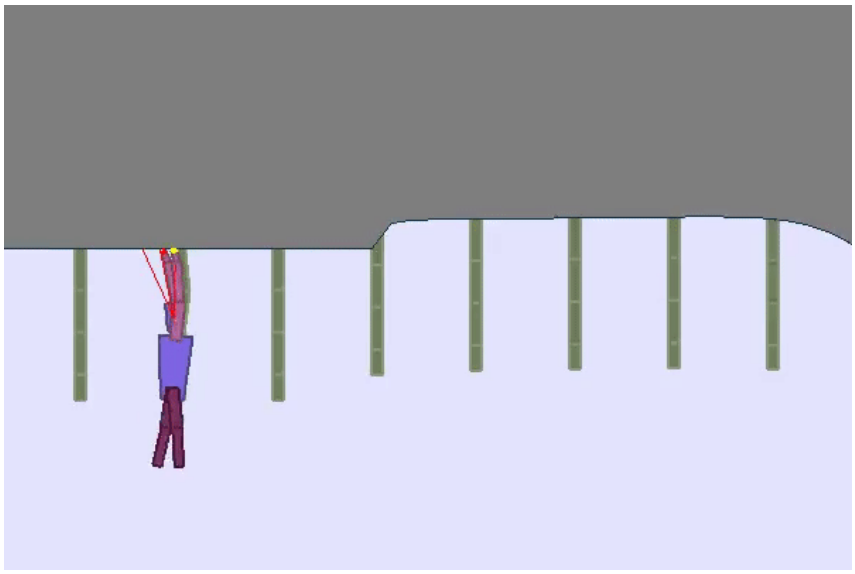


Figure 3: Moving obstacles in the TeachMyAgent benchmark – TeachMyAgent [original image]

In such cases, it seems needed to propose different tasks to our RL agent and organize them such that the agent progressively acquires skills. This approach is called **Curriculum Learning** and usually implies a hand-designed curriculum (or set of tasks organized in a specific order). In practice, one can, for instance, control the generation of the environment, the initial states, or use Self-Play and control the level of opponents proposed to the RL agent.

As designing such a curriculum is not always trivial, the field of **Automatic Curriculum Learning (ACL) proposes to design approaches that learn to create such an organization of tasks in order to maximize the RL agent's performances**. Portelas et al. proposed to define ACL as:

> . . . a family of mechanisms that automatically adapt the distribution of training data by learning to adjust the selection of learning situations to the capabilities of RL agents.

As an example, OpenAI used **Domain Randomization** (they applied random variations on the environment) to make a robot hand solve Rubik's Cubes.



Figure 4: Domain randomization for the robotic Rubik's Cube task – OpenAI - Solving Rubik's Cube with a Robot Hand [original image]

Finally, you can play with the robustness of agents trained in the TeachMyAgent benchmark by controlling environment variations or even drawing the terrain

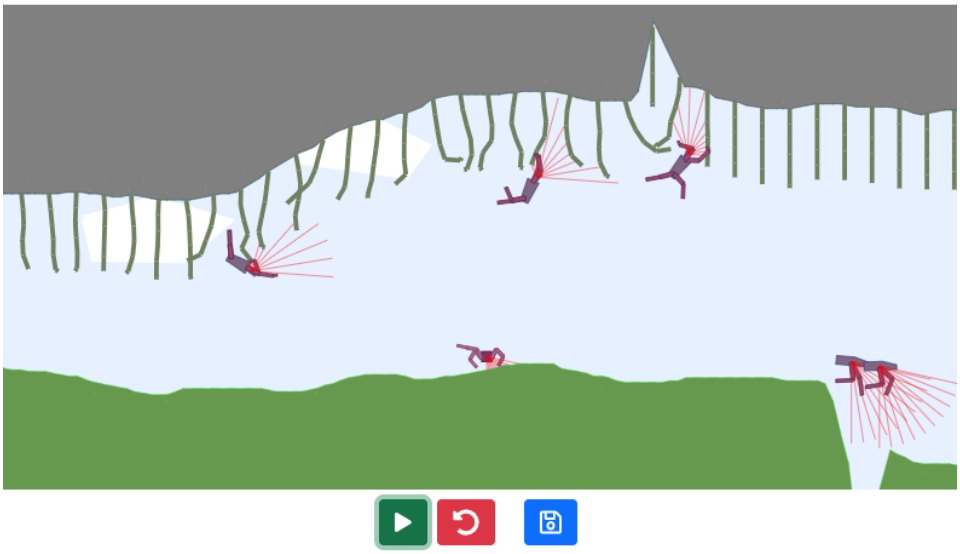


Figure 5: Interactive TeachMyAgent environment and terrain variations – https://huggingface.co/spaces/flowers-team/Interactive_DeepRL_Demo [original image]

2.1 Further reading

For more information, we recommend that you check out the following resources:

Overview of the field

- Automatic Curriculum Learning For Deep RL: A Short Survey
- Curriculum for Reinforcement Learning

Recent methods

- Evolving Curricula with Regret-Based Environment Design

- Curriculum Reinforcement Learning via Constrained Optimal Transport
- Prioritized Level Replay

2.2 Author

This section was written by Clément Romac

3 Generalization in Reinforcement Learning

Generalization plays a pivotal role in the realm of Reinforcement Learning. While **RL algorithms demonstrate good performance in controlled environments**, the real world presents a **unique challenge due to its non-stationary and open-ended nature**.

As a result, the development of RL algorithms that stay robust in the face of environmental variations, coupled with the capability to transfer and adapt to uncharted yet analogous tasks and settings, becomes fundamental for real world application of RL.

If you're interested to dive deeper into this research subject, we recommend exploring the following resource:

- Generalization in Reinforcement Learning by Robert Kirk: this comprehensive survey provides an insightful **overview of the concept of generalization in RL**, making it an excellent starting point for your exploration.
- Improving Generalization in Reinforcement Learning using Policy Similarity Embeddings

4 Model Based Reinforcement Learning (MBRL)

Model-based reinforcement learning only differs from its model-free counterpart in learning a *dynamics model*, but that has substantial downstream effects on how the decisions are made.

The dynamics model usually models the environment transition dynamics, $s_{t+1} = f_\theta(s_t, a_t)$, but things like inverse dynamics models (mapping from states to actions) or reward models (predicting rewards) can be used in this framework.

4.1 Simple definition

- There is an agent that repeatedly tries to solve a problem, **accumulating state and action data**.
- With that data, the agent creates a structured learning tool, *a dynamics model*, to reason about the world.
- With the dynamics model, the agent **decides how to act by predicting the future**.
- With those actions, **the agent collects more data, improves said model, and hopefully improves future actions**.

4.2 Academic definition

Model-based reinforcement learning (MBRL) follows the framework of an agent interacting in an environment, **learning a model of said environment**, and then **leveraging the model for control (making decisions)**.

Specifically, the agent acts in a Markov Decision Process (MDP) governed by a transition function $s_{t+1} = f(s_t, a_t)$ and returns a reward at each step $r(s_t, a_t)$. With a collected dataset $D := s_i, a_i, s_{i+1}, r_i$, the agent learns a model, $s_{t+1} = f_\theta(s_t, a_t)$ **to minimize the negative log-likelihood of the transitions**.

We employ sample-based model-predictive control (MPC) using the learned dynamics model, which optimizes the expected reward over a finite, recursively predicted horizon, τ, from a set of actions sampled from a uniform distribution $U(a)$, (see paper or paper or paper).

4.3 Further reading

For more information on MBRL, we recommend you check out the following resources:

- A blog post on debugging MBRL.
- A recent review paper on MBRL,

4.4 Author

This section was written by Nathan Lambert

5 Offline vs. Online Reinforcement Learning

Deep Reinforcement Learning (RL) is a framework **to build decision-making agents**. These agents aim to learn optimal behavior (policy) by interacting with the environment through **trial and error and receiving rewards as unique feedback**.

The agent's goal **is to maximize its cumulative reward**, called return. Because RL is based on the *reward hypothesis*: all goals can be described as the **maximization of the expected cumulative reward**.

Deep Reinforcement Learning agents **learn with batches of experience**. The question is, how do they collect it?:

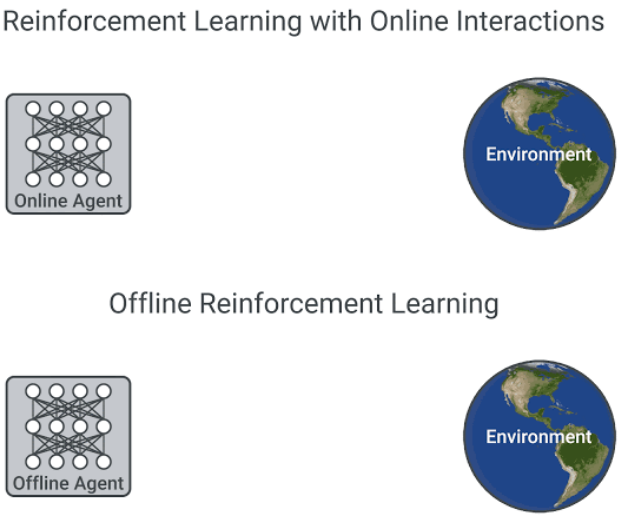


Figure 6: Comparison of offline and online reinforcement learning – A comparison between Reinforcement Learning in an Online and Offline setting, figure taken from: this post [original image]

- In *online reinforcement learning*, which is what we've learned during this course, the agent **gathers data directly**: it collects a batch of experience by **interacting with the environment**. Then, it uses this experience immediately (or via some replay buffer) to learn from it (update its policy).

But this implies that either you **train your agent directly in the real world or have a simulator**. If you don't have one, you need to build it, which can be very complex (how to reflect the complex reality of the real world in an environment?), expensive, and insecure (if the simulator has flaws that may provide a competitive advantage, the agent will exploit them).

- On the other hand, in *offline reinforcement learning*, the agent only **uses data collected from other agents or human demonstrations**. It does **not interact with the environment**.

The process is as follows:

- **Create a dataset** using one or more policies and/or human interactions.
- Run **offline RL on this dataset** to learn a policy

This method has one drawback: the *counterfactual queries problem*. What do we do if our agent **decides to do something for which we don't have the data?** For instance, turning right on an intersection but we don't have this trajectory.

There exist some solutions on this topic, but if you want to know more about offline reinforcement learning, you can watch this video

5.1 Further reading

For more information, we recommend you check out the following resources:

- Offline Reinforcement Learning, Talk by Sergei Levine
- Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems

5.2 Author

This section was written by Thomas Simonini

6 Decision Transformers

The Decision Transformer model was introduced by "Decision Transformer: Reinforcement Learning via Sequence Modeling" by Chen L. et al. It abstracts Reinforcement Learning as a conditional-sequence modeling problem.

The main idea is that instead of training a policy using RL methods, such as fitting a value function, that will tell us what action to take to maximize the return (cumulative reward), **we use a sequence modeling algorithm (Transformer) that, given a desired return, past states, and actions, will generate future actions to achieve this desired return**. It's an autoregressive model conditioned on the desired return, past states, and actions to generate future actions that achieve the desired return.

This is a complete shift in the Reinforcement Learning paradigm since we use generative trajectory modeling (modeling the joint distribution of the sequence of states, actions, and rewards) to replace conventional RL algorithms. This means that in Decision Transformers, we don't maximize the return but rather generate a series of future actions that achieve the desired return.

The Hugging Face Transformers team integrated the Decision Transformer, an Offline Reinforcement Learning method, into the library as well as the Hugging Face Hub.

6.1 Learn about Decision Transformers

To learn more about Decision Transformers, you should read the blogpost we wrote about it Introducing Decision Transformers on Hugging Face

6.2 Train your first Decision Transformers

Now that you understand how Decision Transformers work thanks to Introducing Decision Transformers on Hugging Face, you're ready to learn to train your first Offline Decision Transformer model from scratch to make a half-cheetah run.

Start the tutorial here https://huggingface.co/blog/train-decision-transformers

6.3 Further reading

For more information, we recommend that you check out the following resources:

- Decision Transformer: Reinforcement Learning via Sequence Modeling
- Online Decision Transformer

6.4 Author

This section was written by Edward Beeching

7 RLHF

Reinforcement learning from human feedback (RLHF) is a **methodology for integrating human data labels into a RL-based optimization process**. It is motivated by the **challenge of modeling human preferences**.

For many questions, even if you could try and write down an equation for one ideal, humans differ on their preferences.

Updating models **based on measured data is an avenue to try and alleviate these inherently human ML problems**.

7.1 Start Learning about RLHF

To start learning about RLHF:

1. Read this introduction: Illustrating Reinforcement Learning from Human Feedback (RLHF).
2. Watch the recorded live we did some weeks ago, where Nathan covered the basics of Reinforcement Learning from Human Feedback (RLHF) and how this technology is being used to enable state-of-the-art ML tools like ChatGPT. Most of the talk is an overview of the interconnected ML models. It covers the basics of Natural Language Processing and RL and how RLHF is used on large language models. We then conclude with open questions in RLHF.

Video lecture: Watch on YouTube

3. Read other blogs on this topic, such as Closed-API vs Open-source continues: RLHF, ChatGPT, data moats. Let us know if there are more you like!

7.2 Additional readings

Note, this is copied from the Illustrating RLHF blog post above. Here is a list of the most prevalent papers on RLHF to date. The field was recently popularized with the emergence of DeepRL (around 2017) and has grown into a broader study of the applications of LLMs from many large technology companies. Here are some papers on RLHF that pre-date the LM focus:

- TAMER: Training an Agent Manually via Evaluative Reinforcement (Knox and Stone 2008): Proposed a learned agent where humans provided scores on the actions taken iteratively to learn a reward model.
- Interactive Learning from Policy-Dependent Human Feedback (MacGlashan et al. 2017): Proposed an actor-critic algorithm, COACH, where human feedback (both positive and negative) is used to tune the advantage function.
- Deep Reinforcement Learning from Human Preferences (Christiano et al. 2017): RLHF applied on preferences between Atari trajectories.

- Deep TAMER: Interactive Agent Shaping in High-Dimensional State Spaces (Warnell et al. 2018): Extends the TAMER framework where a deep neural network is used to model the reward prediction.

And here is a snapshot of the growing set of papers that show RLHF's performance for LMs:

- Fine-Tuning Language Models from Human Preferences (Zieglar et al. 2019): An early paper that studies the impact of reward learning on four specific tasks.
- Learning to summarize with human feedback (Stiennon et al., 2020): RLHF applied to the task of summarizing text. Also, Recursively Summarizing Books with Human Feedback (OpenAI Alignment Team 2021), follow on work summarizing books.
- WebGPT: Browser-assisted question-answering with human feedback (OpenAI, 2021): Using RLHF to train an agent to navigate the web.
- InstructGPT: Training language models to follow instructions with human feedback (OpenAI Alignment Team 2022): RLHF applied to a general language model [Blog post on InstructGPT].
- GopherCite: Teaching language models to support answers with verified quotes (Menick et al. 2022): Train a LM with RLHF to return answers with specific citations.
- Sparrow: Improving alignment of dialogue agents via targeted human judgements (Glaese et al. 2022): Fine-tuning a dialogue agent with RLHF
- ChatGPT: Optimizing Language Models for Dialogue (OpenAI 2022): Training a LM with RLHF for suitable use as an all-purpose chat bot.
- Scaling Laws for Reward Model Overoptimization (Gao et al. 2022): studies the scaling properties of the learned preference model in RLHF.
- Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback (Anthropic, 2022): A detailed documentation of training a LM assistant with RLHF.
- Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned (Ganguli et al. 2022): A detailed documentation of efforts to "discover, measure, and attempt to reduce [language models] potentially harmful outputs."
- Dynamic Planning in Open-Ended Dialogue using Reinforcement Learning (Cohen at al. 2022): Using RL to enhance the conversational skill of an open-ended dialogue agent.
- Is Reinforcement Learning (Not) for Natural Language Processing?: Benchmarks, Baselines, and Building Blocks for Natural Language Policy Optimization (Ramamurthy and Ammanabrolu et al. 2022): Discusses the design space of open-source tools in RLHF and proposes a new algorithm NLPO (Natural Language Policy Optimization) as an alternative to PPO.

7.3 Author

This section was written by Nathan Lambert

8 Language models in RL

8.1 LMs encode useful knowledge for agents

Language models (LMs) can exhibit impressive abilities when manipulating text such as question-answering or even step-by-step reasoning. Additionally, their training on massive text corpora allowed them to **encode various types of knowledge including abstract ones about the physical rules of our world** (for instance what is possible to do with an object, what happens when one rotates an object...).

A natural question recently studied was whether such knowledge could benefit agents such as robots when trying to solve everyday tasks. And while these works showed interesting results, the proposed agents lacked any learning method. **This limitation prevents these agent from adapting to the environment (e.g. fixing wrong knowledge) or learning new skills.**

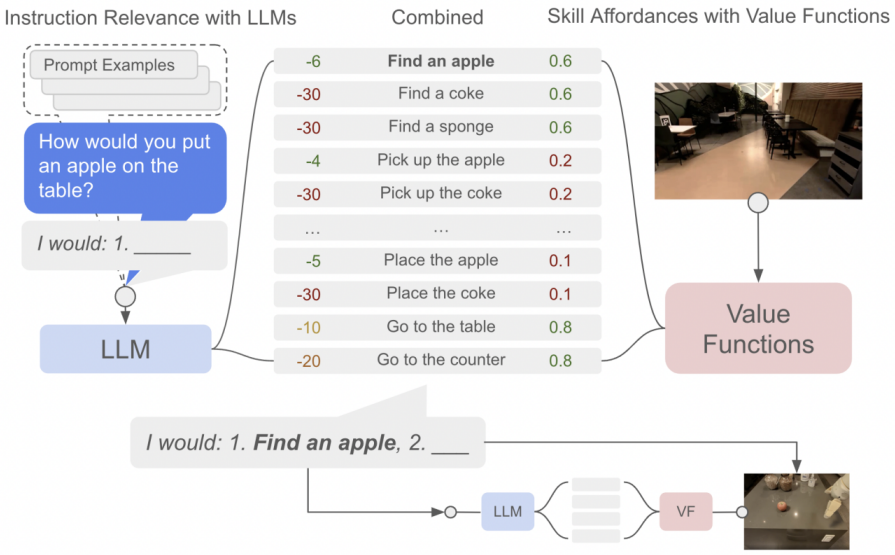


Figure 7: Grounding language-model knowledge in robotic affordances – Source: Towards Helpful Robots: Grounding Language in Robotic Affordances [original image]

8.2 LMs and RL

There is therefore a potential synergy between LMs which can bring knowledge about the world, and RL which can align and correct this knowledge by interacting with an environment. It is especially interesting from a RL point-of-view as the RL field mostly relies on the **Tabula-rasa** setup where everything is learned from scratch by the agent leading to:

1) Sample inefficiency
2) Unexpected behaviors from humans' eyes

As a first attempt, the paper "Grounding Large Language Models with Online Reinforcement Learning" tackled the problem of **adapting or aligning a LM to a textual environment using PPO**. They showed that the knowledge encoded in the LM lead to a fast adaptation to the environment (opening avenues for sample efficient RL agents) but also that such knowledge allowed the LM to better generalize to new tasks once aligned.

Video demonstration: Open original video

Another direction studied in "Guiding Pretraining in Reinforcement Learning with Large Language Models" was to keep the LM frozen but leverage its knowledge to **guide an RL agent's exploration**. Such a method allows the RL agent to be guided towards human-meaningful and plausibly useful behaviors without requiring a human in the loop during training.

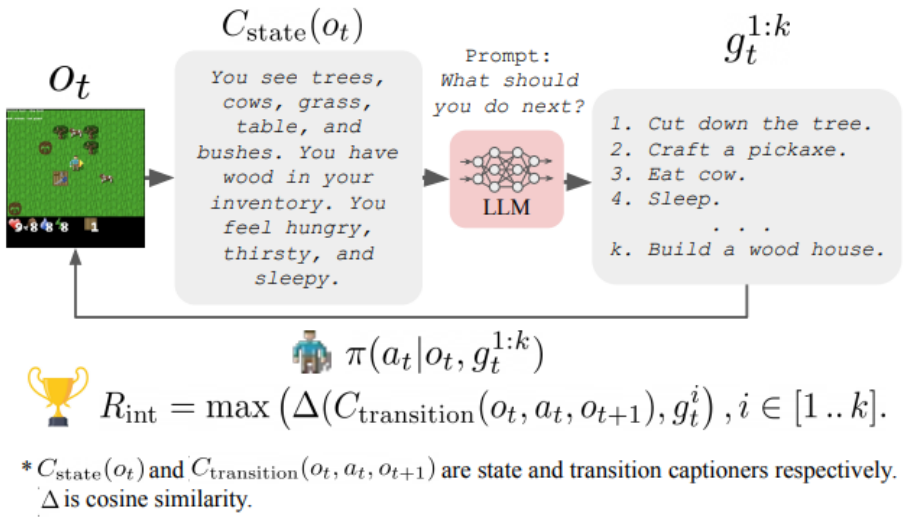


Figure 8: Using language-model knowledge to guide reinforcement-learning agents – Source: Towards Helpful Robots: Grounding Language in Robotic Affordances [original image]

Several limitations make these works still very preliminary such as the need to convert the agent's observation to text before giving it to a LM as well as the compute cost of interacting with very large LMs.

8.3 Further reading

For more information we recommend you check out the following resources:

- Google Research, 2022 & beyond: Robotics
- Pre-Trained Language Models for Interactive Decision-Making
- Grounding Large Language Models with Online Reinforcement Learning
- Guiding Pretraining in Reinforcement Learning with Large Language Models

8.4 Author

This section was written by Clément Romac

9 An Introduction to Unreal Learning Agents

Learning Agents is an Unreal Engine (UE) plugin that allows you **to train AI characters using machine learning (ML) in Unreal**.

It's an exciting new plugin where you can create unique environments using Unreal Engine and train your agents.

Let's see how you can **get started and train a car to drive in an Unreal Engine Environment**.



Figure 9: Unreal Engine Learning Agents: driving-agent tutorial – Source: Learning Agents Driving Car Tutorial [original image]

9.1 Case 1: I don't know anything about Unreal Engine and Beginners in Unreal Engine

If you're new to Unreal Engine, don't be scared! We listed two courses you need to follow to be able to use Learning Agents:

1. Master the Basics: Begin by watching this course your first hour in Unreal Engine 5. This comprehensive course will **lay down the foundational knowledge you need to use Unreal**.
2. Dive into Blueprints: Explore the world of Blueprints, the visual scripting component of Unreal Engine. This video course will familiarize you with this essential tool.

Armed with the basics, **you're now prepared to play with Learning Agents**:

3. Get the Big Picture of Learning Agents by reading this informative overview.
4. Teach a Car to Drive using Reinforcement Learning in Learning Agents.
5. Check Imitation Learning with the Unreal Engine 5.3 Learning Agents Plugin

9.2 Case 2: I'm familiar with Unreal

For those already acquainted with Unreal Engine, you can jump straight into Learning Agents with these two tutorials:

1. Get the Big Picture of Learning Agents by reading this informative overview.
2. Teach a Car to Drive using Reinforcement Learning in Learning Agents. .
3. Check Imitation Learning with the Unreal Engine 5.3 Learning Agents Plugin

10 Godot RL Agents

Godot RL Agents is an Open Source package that allows video game creators, AI researchers, and hobbyists the opportunity **to learn complex behaviors for their Non Player Characters or agents**.

The library provides:

- An interface between games created in the Godot Engine and Machine Learning algorithms running in Python
- Wrappers for four well known rl frameworks: StableBaselines3, CleanRL, Sample Factory and Ray RLLib
- Support for memory-based agents with LSTM or attention based interfaces
- Support for *2D and 3D games*
- A suite of *AI sensors* to augment your agent's capacity to observe the game world
- Godot and Godot RL Agents are **completely free and open source under a very permissive MIT license**. No strings attached, no royalties, nothing.

You can find out more about Godot RL agents on their GitHub page or their AAAI-2022 Workshop paper. The library's creator, Ed Beeching, is a Research Scientist here at Hugging Face.

Installation of the library is simple: `pip install godot-rl`

10.1 Create a custom RL environment with Godot RL Agents

In this section, you will **learn how to create a custom environment in the Godot Game Engine** and then implement an AI controller that learns to play with Deep Reinforcement Learning.

The example game we create today is simple, **but shows off many of the features of the Godot Engine and the Godot RL Agents library**. You can then dive into the examples for more complex environments and behaviors.

The environment we will be building today is called Ring Pong, the game of pong but the pitch is a ring and the paddle moves around the ring. The **objective is to keep the ball bouncing inside the ring**.

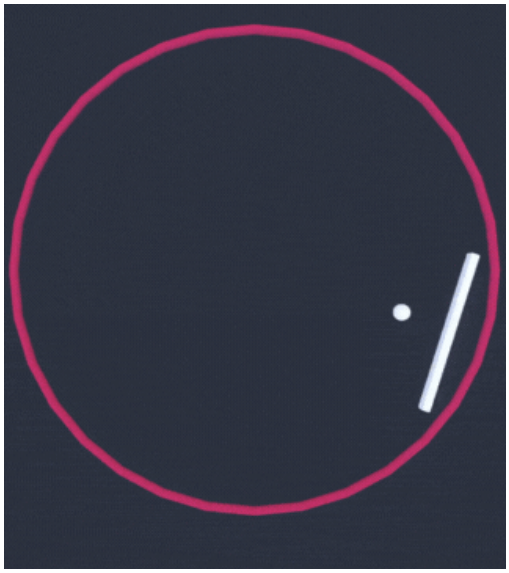


Figure 10: Ring Pong: a custom environment built in Godot [original image]

Installing the Godot Game Engine

The Godot game engine is an open source tool for the **creation of video games, tools and user interfaces**.

Godot Engine is a feature-packed, cross-platform game engine designed to create 2D and 3D games from a unified interface. It provides a comprehensive set of common tools, so users **can focus on making games without having to reinvent the wheel**. Games can be exported in one click to a number of platforms, including the major desktop platforms (Linux, macOS, Windows) as well as mobile (Android, iOS) and web-based (HTML5) platforms.

While we will guide you through the steps to implement your agent, you may wish to learn more about the Godot Game Engine. Their documentation is thorough, and there are many tutorials on YouTube we would also recommend GDQuest, KidsCanCode and Bramwell as sources of information.

In order to create games in Godot, **you must first download the editor**. Godot RL Agents supports the latest version of Godot, Godot 4.0.

Which can be downloaded at the following links:

- Windows
- Mac
- Linux

Loading the starter project

We provide two versions of the codebase:

- A starter project, to download and follow along for this tutorial
- A final version of the project, for comparison and debugging.

To load the project, in the Godot Project Manager click **Import**, navigate to where the files are located and load the **project.godot** file.

If you press F5 or play in the editor, you should be able to play the game in human mode. There are several instances of the game running, this is because we want to speed up training our AI agent with many parallel environments.

Installing the Godot RL Agents plugin

The Godot RL Agents plugin can be installed from the Github repo or with the Godot Asset Lib in the editor.

First click on the AssetLib and search for “rl”

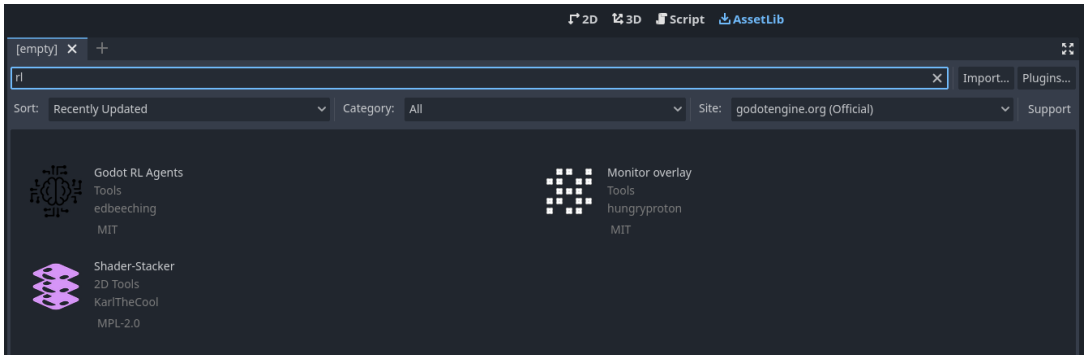


Figure 11: Godot AssetLib: searching for the RL Agents plugin [original image]

Then click on Godot RL Agents, click Download and unselect the LICENSE and README .md files. Then click install.

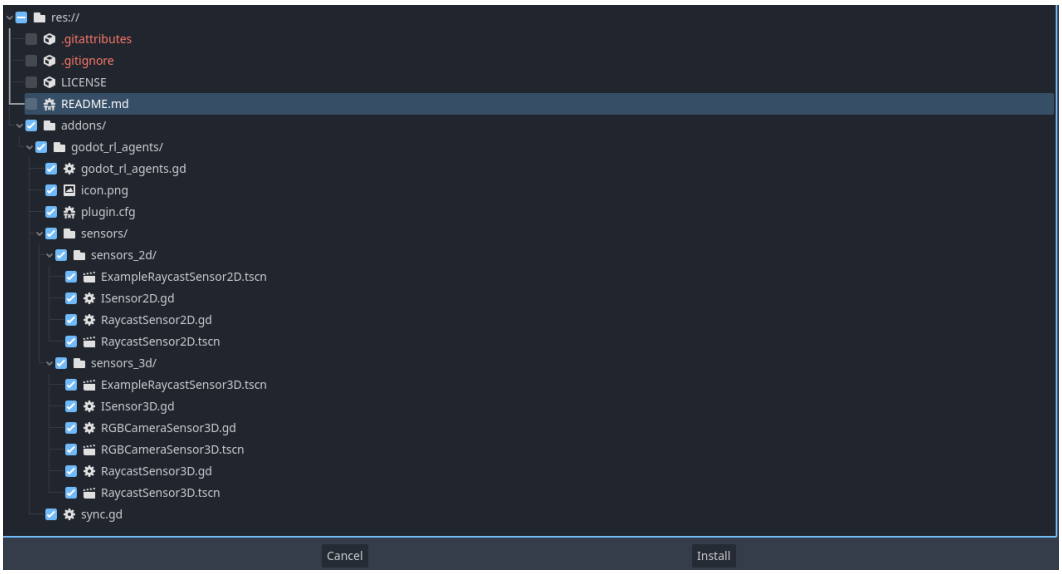


Figure 12: Godot: installing the RL Agents plugin [original image]

The Godot RL Agents plugin is now downloaded to your machine. Now click on Project -> Project settings and enable the addon:

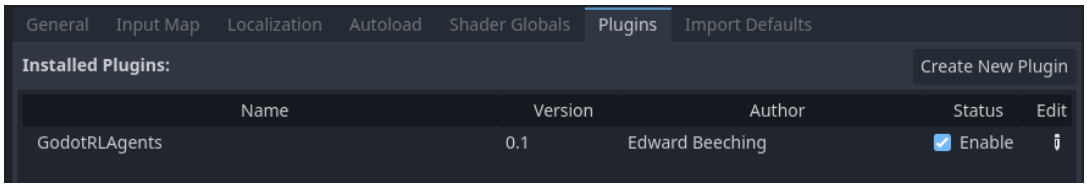


Figure 13: Godot project settings: enabling the RL Agents add-on [original image]

Adding the AI controller

We now want to add an AI controller to our game. Open the player.tscn scene, on the left you should see a hierarchy of nodes that looks like this:

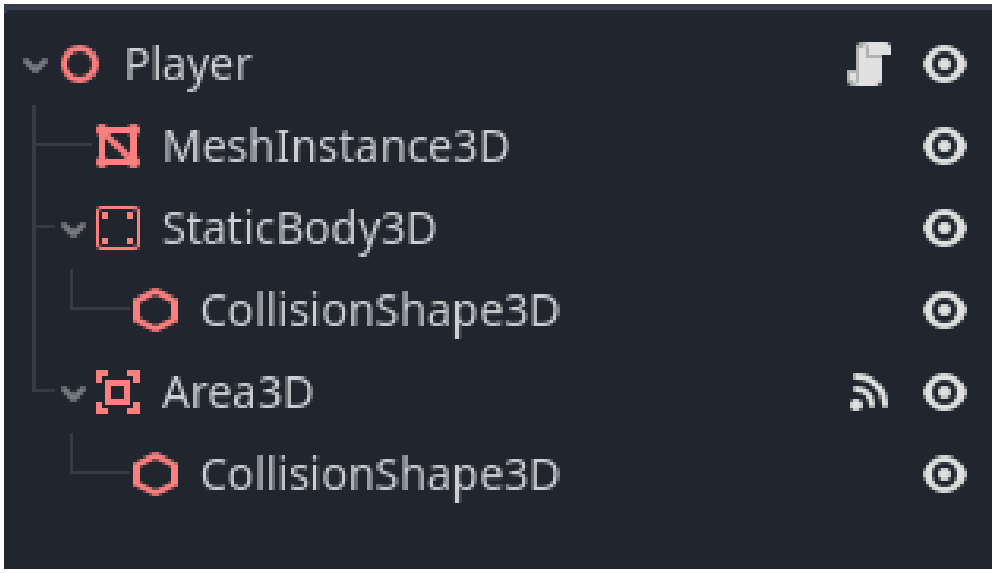


Figure 14: Godot scene tree before adding the AI controller [original image]

Right click the **Player** node and click **Add Child Node.** There are many nodes listed here, search for AIController3D and create it.

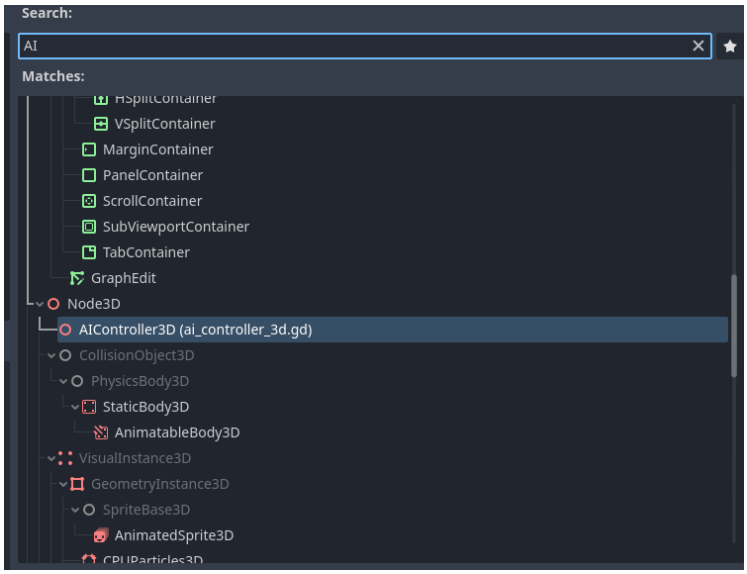


Figure 15: Godot: adding an AIController3D child node [original image]

The AI Controller Node should have been added to the scene tree, next to it is a scroll. Click on it to open the script that is attached to the AIController. The Godot game engine uses a scripting language called GDScript, which is syntactically similar to python. The script contains methods that need to be implemented in order to get our AI controller working.

```
#-- Methods that need implementing using the "extend script" option in Godot --#
func get_obs() -> Dictionary:
    assert(false, "the get_obs method is not implemented when extending from ai_controller")
    return {"obs":[]}

func get_reward() -> float:
    assert(false, "the get_reward method is not implemented when extending from ai_controller")
    return 0.0

func get_action_space() -> Dictionary:
    assert(false, "the get get_action_space method is not implemented when extending from
    ai_controller")
    return {
        "example_actions_continous" : {
            "size": 2,
            "action_type": "continuous"
        },
        "example_actions_discrete" : {
            "size": 2,
            "action_type": "discrete"
        },
        }

func set_action(action) -> void:
    assert(false, "the get set_action method is not implemented when extending from ai_controller")
# -----------------------------------------------------------------------------#
```

In order to implement these methods, we will need to create a class that inherits from AIController3D. This is easy to do in Godot, and is called “extending” a class.

Right click the AIController3D Node and click “Extend Script” and call the new script `controller.gd`. You should now have an almost empty script file that looks like this:

```
extends AIController3D

# Called when the node enters the scene tree for the first time.
func _ready():
```

```
    pass # Replace with function body.

# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta):
    pass
```

We will now implement the 4 missing methods, delete this code, and replace it with the following:

```
extends AIController3D

# Stores the action sampled for the agent's policy, running in python
var move_action : float = 0.0

func get_obs() -> Dictionary:
    # get the balls position and velocity in the paddle's frame of reference
    var ball_pos = to_local(_player.ball.global_position)
    var ball_vel = to_local(_player.ball.linear_velocity)
    var obs = [ball_pos.x, ball_pos.z, ball_vel.x/10.0, ball_vel.z/10.0]

    return {"obs":obs}

func get_reward() -> float:
    return reward

func get_action_space() -> Dictionary:
    return {
        "move_action" : {
            "size": 1,
            "action_type": "continuous"
        },
        }

func set_action(action) -> void:
    move_action = clamp(action["move_action"][0], -1.0, 1.0)
```

We have now defined the agent's observation, which is the position and velocity of the ball in its local coordinate space. We have also defined the action space of the agent, which is a single continuous value ranging from -1 to +1.

The next step is to update the Player's script to use the actions from the AIController, edit the Player's script by clicking on the scroll next to the player node, update the code in `Player.gd` to the following:

```
extends Node3D

@export var rotation_speed = 3.0
@onready var ball = get_node("../Ball")
@onready var ai_controller = $AIController3D

func _ready():
    ai_controller.init(self)

func game_over():
    ai_controller.done = true
    ai_controller.needs_reset = true

func _physics_process(delta):
    if ai_controller.needs_reset:
        ai_controller.reset()
        ball.reset()
        return
```

```
    var movement : float
    if ai_controller.heuristic == "human":
        movement = Input.get_axis("rotate_anticlockwise", "rotate_clockwise")
    else:
        movement = ai_controller.move_action
    rotate_y(movement*delta*rotation_speed)

func _on_area_3d_body_entered(body):
    ai_controller.reward += 1.0
```

We now need to synchronize between the game running in Godot and the neural network being trained in Python. Godot RL agents provides a node that does just that. Open the train.tscn scene, right click on the root node, and click "Add child node". Then, search for "sync" and add a Godot RL Agents Sync node. This node handles the communication between Python and Godot over TCP.

You can run training live in the editor, by first launching the python training with `gdrl`.

In this simple example, a reasonable policy is learned in several minutes. You may wish to speed up training, click on the Sync node in the train scene, and you will see there is a "Speed Up" property exposed in the editor:

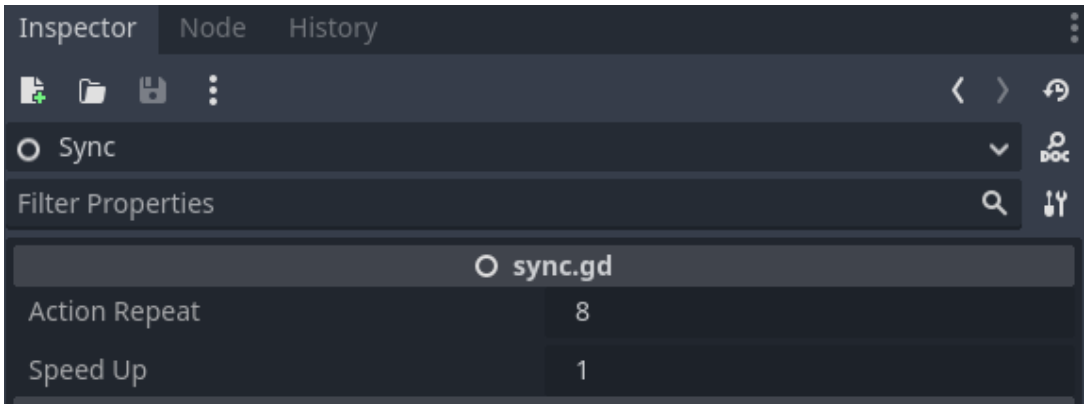


Figure 16: Godot RL Agents Sync node: Speed Up property [original image]

Try setting this property up to 8 to speed up training. This can be a great benefit on more complex environments, like the multi-player FPS we will learn about in the next chapter.

Export model

Reference doc

Let's put aside the Godot editor for now. We'll need to use terminals to run some commands in order to save models we trained.

The latest version of the Godot RL library provides experimental support for onnx models with the Stable Baselines 3, rllib, and CleanRL training frameworks.

For example, let's use the Stable Baselines 3 as the framework. Train your agent using the sb3 example (instructions for using the script), enabling the option `--onnx_export_path=model.onnx`

Below is an example command line to execute:

```
cd <....> # go into this Godot project directory
python stable_baselines3_example.py --timesteps=100_000 --onnx_export_path=model.onnx --
    save_model_path=model.zip --save_checkpoint_frequency=20_000 --experiment_name=exp1
```

If things work correctly, you should see messages printed out in the terminal like below:

```
No game binary has been provided, please press PLAY in the Godot editor
waiting for remote GODOT connection on port 11008
```

If you encounter failures about import error in stable_baselines3_example script: "ImportError: cannot import name 'export_model_as_onnx' from 'godot_rl.wrappers.onnx.stable_baselines_export' ", follow the answer in this issue here.

Now it's time to switch back to the Godot editor, and hit PLAY on the top right corner. Once you hit that, the game scene will pop up showing AI training. In the meantime, the terminal will start to print out metrics. Wait for the training to finish, and if things work correctly, you should be able to find the file `model.onnx` in the Godot project directory.

Apply AI in the game!

Now let's apply this trained model to the game!

In the Godot editor, find the Sync node in `train.tscn`:

- change the control mode to `Onnx Inference` from the dropdown
- set `Onnx Model Path` to the model file name, in our case here it's `model.onnx`

To run this game, we need the mono version (i.e., the .NET version) of the Godot editor, you can download it from the Godot official page. We need to install .NET as well.

Most likely you wil encounter errors in the first attempt. Below are the scenarios to help you resolve the errors.

1. issue about `Invalid Call. Nonexistent function 'new' in base 'CSharpScript'`: solution
2. errors about `onnxruntime` on MacOS: solution

There's more!

We have only scratched the surface of what can be achieved with Godot RL Agents, the library includes custom sensors and cameras to enrich the information available to the agent. Take a look at the examples to find out more!

For the ability to export the trained model to .onnx so that you can run inference directly from Godot without the Python server, and other useful training options, take a look at the advanced SB3 tutorial.

10.2 Author

This section was written by Edward Beeching

11 Brief introduction to RL documentation

In this advanced topic, we address the question: **how should we monitor and keep track of powerful reinforcement learning agents that we are training in the real world and interfacing with humans?**

As machine learning systems have increasingly impacted modern life, the **call for the documentation of these systems has grown**.

Such documentation can cover aspects such as the training data used – where it is stored, when it was collected, who was involved, etc. – or the model optimization framework – the architecture, evaluation metrics, relevant papers, etc. – and more.

Today, model cards and datasheets are becoming increasingly available. For example, on the Hub (see documentation here).

If you click on a popular model on the Hub, you can learn about its creation process.

These model and data specific logs are designed to be completed when the model or dataset are created, leaving them to go un-updated when these models are built into evolving systems in the future.

11.1 Motivating Reward Reports

Reinforcement learning systems are fundamentally designed to optimize based on measurements of reward and time. While the notion of a reward function can be mapped nicely to many well-understood fields of supervised learning (via a loss function), understanding of how machine learning systems evolve over time is limited.

To that end, the authors introduce *Reward Reports for Reinforcement Learning* (the pithy naming is designed to mirror the popular papers *Model Cards for Model Reporting* and *Datasheets for Datasets*). The goal is to propose a type of documentation focused on the **human factors of reward** and **time-varying feedback systems**.

Building on the documentation frameworks for model cards and datasheets proposed by Mitchell et al. and Gebru et al., we argue the need for Reward Reports for AI systems.

Reward Reports are living documents for proposed RL deployments that demarcate design choices.

However, many questions remain about the applicability of this framework to different RL applications, roadblocks to system interpretability, and the resonances between deployed supervised machine learning systems and the sequential decision-making utilized in RL.

At a minimum, Reward Reports are an opportunity for RL practitioners to deliberate on these questions and begin the work of deciding how to resolve them in practice.

11.2 Capturing temporal behavior with documentation

The core piece specific to documentation designed for RL and feedback-driven ML systems is a *change-log*. The change-log updates information from the designer (changed training parameters, data, etc.) along with noticed changes from the user (harmful behavior, unexpected responses, etc.).

The change log is accompanied by update triggers that encourage monitoring these effects.

11.3 Contributing

Some of the most impactful RL-driven systems are multi-stakeholder in nature and behind the closed doors of private corporations. These corporations are largely without regulation, so the burden of documentation falls on the public.

If you are interested in contributing, we are building Reward Reports for popular machine learning systems on a public record on GitHub.

For further reading, you can visit the Reward Reports paper or look an example report.

11.4 Author

This section was written by Nathan Lambert

12 Interesting Environments to try

Here we provide a list of interesting environments you can try to train your agents on:

12.1 DIAMBRA Arena

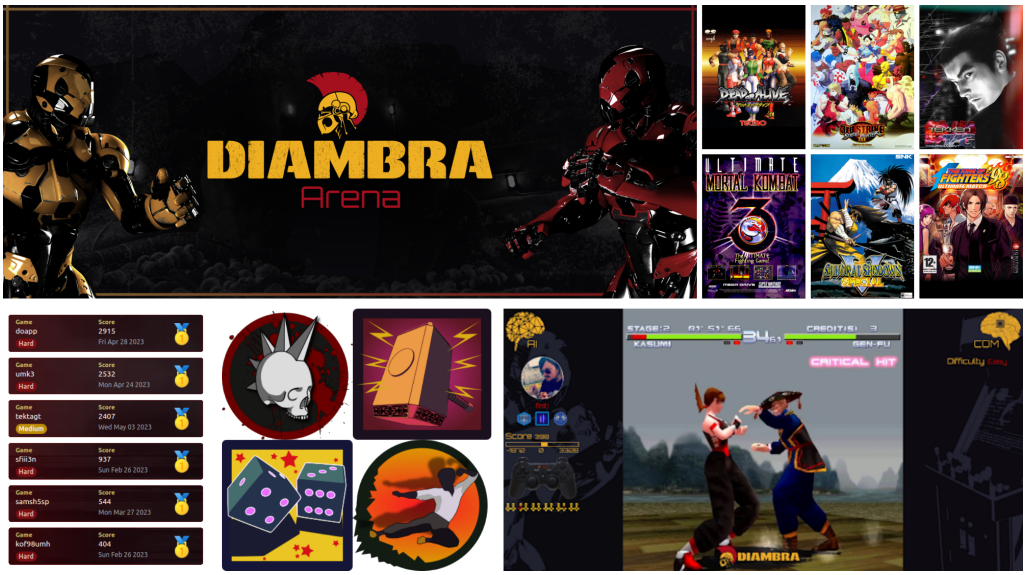


Figure 17: DIAMBRA Arena: arcade environments for reinforcement learning [original image]

DIAMBRA Arena is a software package featuring a collection of high-quality environments for Reinforcement Learning research and experimentation. It provides a standard interface to popular arcade emulated video games, offering a Python API fully compliant with OpenAI Gym/Gymnasium format, that makes its adoption smooth and straightforward.

It supports all major Operating Systems (Linux, Windows and MacOS) and can be easily installed via Python PIP. It is completely free to use, the user only needs to register on the official website.

In addition, its GitHub repository provides a collection of examples covering main use cases of interest that can be run in just a few steps.

Main Features All environments are episodic Reinforcement Learning tasks, with discrete actions (gamepad buttons) and observations composed by screen pixels plus additional numerical data (RAM values like characters health bars or characters stage side).

They all support both single player (1P) as well as two players (2P) mode, making them the perfect resource to explore Standard RL, Competitive Multi-Agent, Competitive Human-Agent, Self-Play, Imitation Learning and Human-in-the-Loop.

Interfaced games have been selected among the most popular fighting retro-games. While sharing the same fundamental mechanics, they provide different challenges, with specific features such as different type and number of characters, how to perform combos, health bars recharging, etc.

DIAMBRA Arena is built to maximize compatibility will all major Reinforcement Learning libraries. It natively provides interfaces with the two most important packages: Stable Baselines 3 and Ray RLlib, while Stable Baselines is also available but deprecated. Their usage is illustrated in the official documentation and in the DIAMBRA Agents examples repository. It can easily be interfaced with any other package in a similar way.

Competition Platform

DIAMBRA also provides a competition platform fully integrated with the Hugging Face Hub, on which you can submit your trained agents and compete with other coders around the globe in epic video games tournaments!

It features a public leaderboard where users are ranked by the best score achieved by their agents in our different environments.

It also offers the possibility to unlock cool achievements depending on the performances of your agent.

Submitted agents are evaluated and their episodes are streamed on DIAMBRA Twitch channel.

References To start using this environment, check these resources:

- Official Docs
- Competition Platform
- GitHub
- Discord

12.2 MineRL

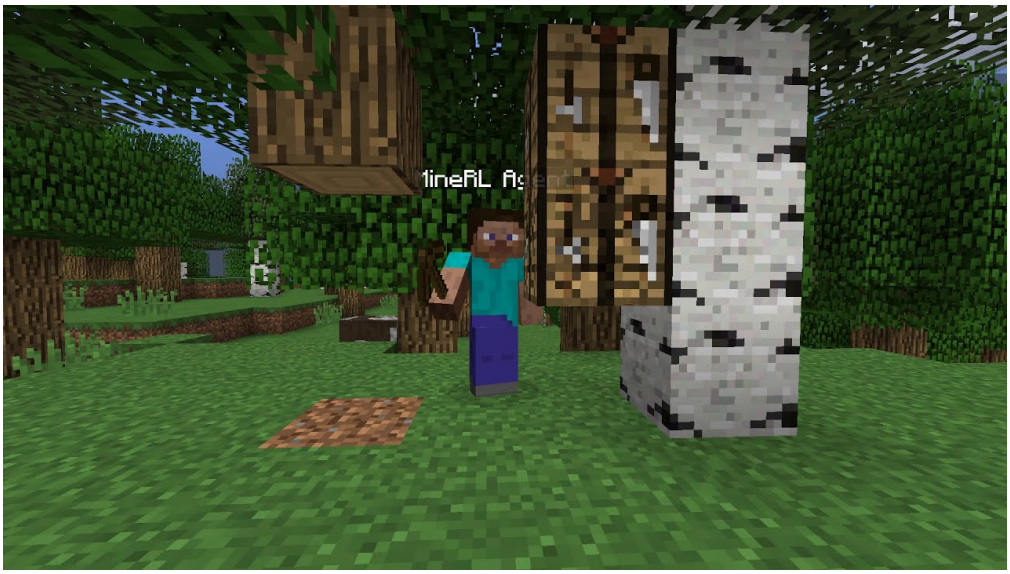


Figure 18: MineRL: Minecraft-based reinforcement learning tasks [original image]

MineRL is a Python library that provides a Gym interface for interacting with the video game Minecraft, accompanied by datasets of human gameplay. Every year there are challenges with this library. Check the website

To start using this environment, check these resources:

- What is MineRL?
- First steps in MineRL
- MineRL documentation and tutorials

12.3 DonkeyCar Simulator

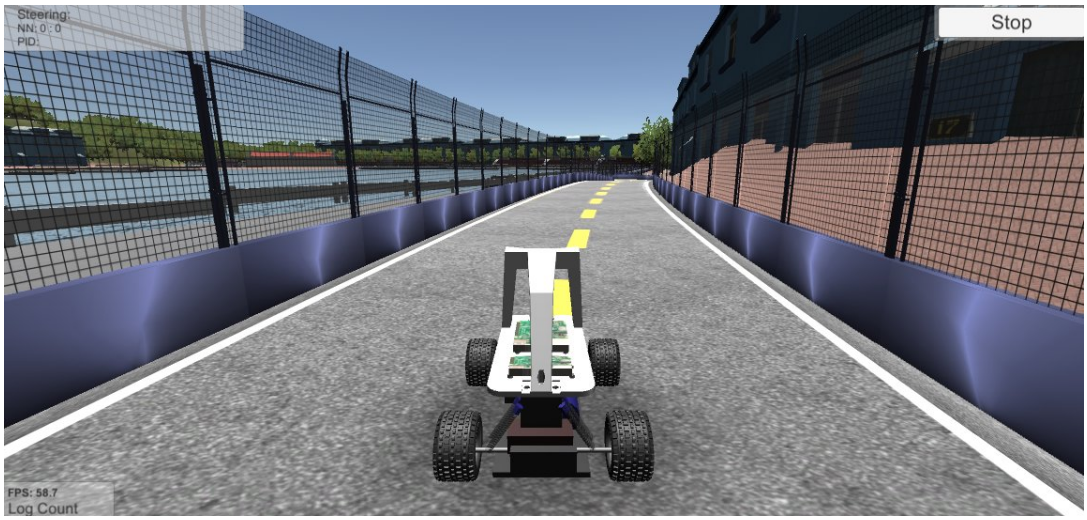


Figure 19: DonkeyCar simulation environment [original image]

Donkey is a Self Driving Car Platform for hobby remote control cars. This simulator version is built on the Unity game platform. It uses their internal physics and graphics and connects to a donkey Python process to use our trained model to control the simulated Donkey (car).

To start using this environment, check these resources:

- DonkeyCar Simulator documentation
- Learn to Drive Smoothly (Antonin Raffin's tutorial) Part 1
- Learn to Drive Smoothly (Antonin Raffin's tutorial) Part 2
- Learn to Drive Smoothly (Antonin Raffin's tutorial) Part 3
- Pretrained agents:
  - https://huggingface.co/araffin/tqc-donkey-mountain-track-v0
  - https://huggingface.co/araffin/tqc-donkey-avc-sparkfun-v0
  - https://huggingface.co/araffin/tqc-donkey-minimonaco-track-v0

12.4 Starcraft II



Figure 20: AlphaStar and StarCraft II as reinforcement learning environments [original image]

Starcraft II is a famous *real-time strategy game*. DeepMind has used this game for their Deep Reinforcement Learning research with Alphastar

To start using this environment, check these resources:

- Starcraft gym
- A. I. Learns to Play Starcraft 2 (Reinforcement Learning) tutorial

12.5 Author

This section was written by Thomas Simonini

13 Student Works

Since the launch of the Deep Reinforcement Learning Course, **many students have created amazing projects that you should check out and consider participating in**.

If you've created an interesting project, don't hesitate to add it to this list by opening a pull request on the GitHub repository.

The projects are **arranged based on the date of publication in this page**.

13.1 Space Scavanger AI

This project is a space game environment with trained neural network for AI.

AI is trained by Reinforcement learning algorithm based on UnityMLAgents and RLlib frameworks.

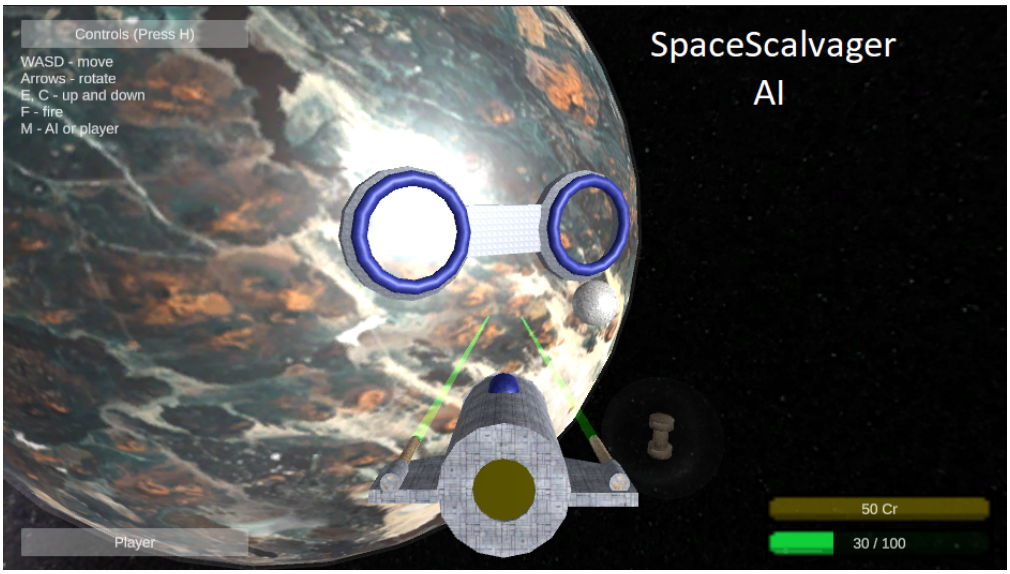


Figure 21: Student project: Space Scavanger AI [original image]

Play the Game here https://swingshuffle.itch.io/spacescalvagerai

Check the Unity project here https://github.com/HighExecutor/SpaceScalvagerAI

13.2 Neural Nitro

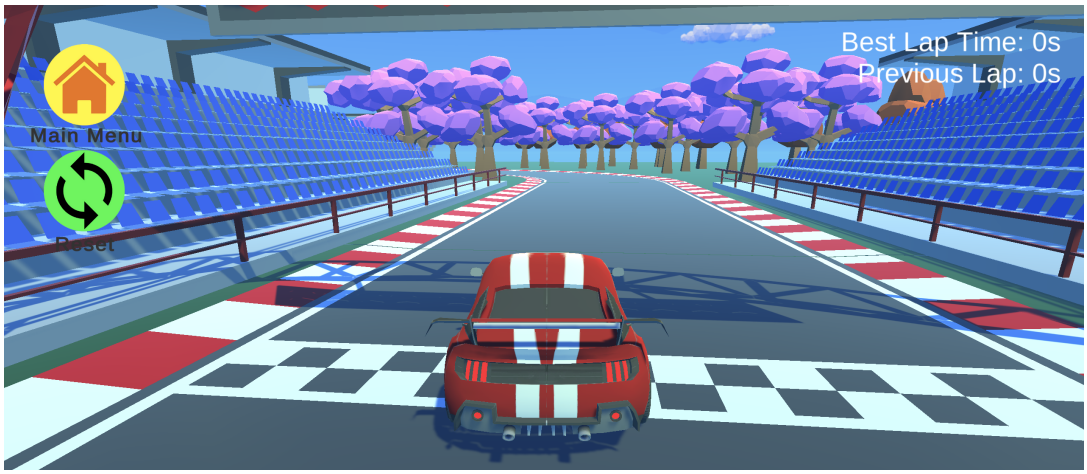


Figure 22: Student project: Neural Nitro racing environment [original image]

In this project, Sookeyy created a low poly racing game and trained a car to drive.

Check out the demo here https://sookeyy.itch.io/neuralnitro

13.3 Space War



Figure 23: Student project: Space War [original image]

In this project, Eric Dong recreates Bill Seiler's 1985 version of Space War in Pygame and uses reinforcement learning (RL) to train AI agents.

This project is currently in development!

Demo

Dev/Edge version:

- https://e-dong.itch.io/spacewar-dev

Stable version:

- https://e-dong.itch.io/spacewar
- https://huggingface.co/spaces/EricofRL/SpaceWarRL

Community blog posts

TBA

Other links

Check out the source here https://github.com/e-dong/space-war-rl
Check out his blog here https://dev.to/edong/space-war-rl-0-series-introduction-25dh

13.4 Decision Transformers for Trading

In this project, student has explored training a Decision Transformer for stock trading. In phase-1, offline training has been implemented. He intends to incorporate online fine-tuning in the next version.

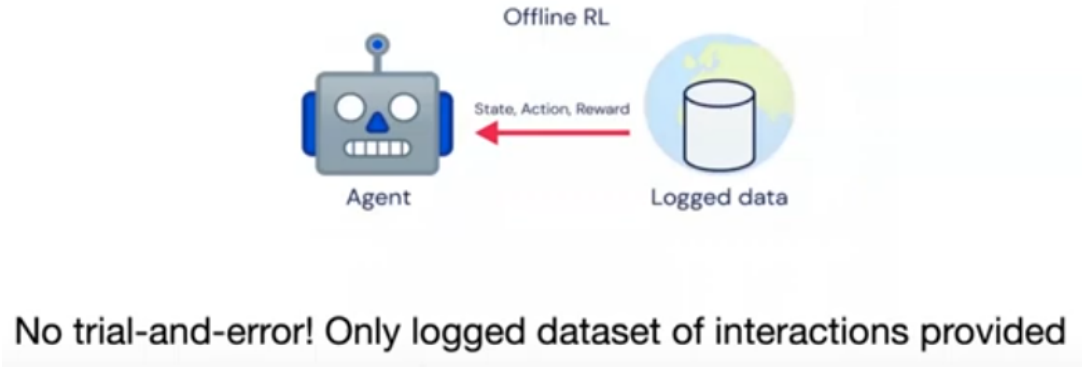


Figure 24: Student project: Decision Transformers for Trading – Source: Stanford CS25: V1 I Decision Transformer: Reinforcement Learning via Sequence Modeling [original image]

Check out the source here https://github.com/ra9hur/Decision-Transformers-For-Trading